

ContourNet: Contour-Based Long-Range Manifold Feature Extraction for Point Cloud Analysis

Anonymous CVPR 2021 submission

Paper ID 5602

Abstract

Recently, the convolution methodology has been adapted into point cloud analysis. This adaptation has outstanding performance in modeling the local structures. However, in terms of characterizing the long-range manifold structures (e.g., contour), the existing models are still too weak to extract the non-local features. Lack of long-range patterns limits the semantic understanding, and this flaw further leads to the low robustness for large geometric transformations. To address this issue, we propose ContourNet, which draws long-range geometric features (i.e., contours) by our proposed viewpoint path mechanism. Specifically, there are three modules in the pipeline. First, to enhance the point features, the point set goes through the Point Encoding Module (PEM). Second, our Viewpoint Encoding Module (VEM) encodes the local features for viewpoints, which are pre-positioned around the point cloud. Third, our Contour-Aware Recurrent Module (CARM) is leveraged to sequentially process the viewpoint features along multiple viewpoint paths. This path mechanism simulates the connectivity nature of point clouds. Thus, regarding the global features, compared with the simple pooling operations, our model can abstract more semantic long-range geometric features (e.g., periodicity of changes in curvature). After extracting the contour features, we gather them into a global descriptor for downstream tasks. Besides, our model has achieved the state-of-the-art performance on the major benchmark datasets, and performs strong robustness against large geometric transformations (e.g., rotation, shear).

1. Introduction

There have emerged lots of promising industries, such as autonomous driving [24], robot manipulation [33] and reverse engineering [44]. These industries extremely put the three-dimensional data (a.k.a. 3D data) at a premium. As an important 3D data format, the point cloud could be treated

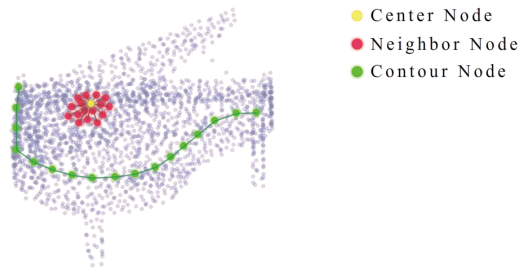


Figure 1. **Local and Contour-Based Connections** (Best view in color and zoom in). We allocate the same node number for these two connections. The local connection consists of a center node and its neighbor nodes, while the contour-based connection is formed by a sequence of nodes. Obviously, the contour-based connection allows the model to cover a further-range manifold with the equal node number. Hence, our contour-based connection can provide more global (i.e., long-range) contexts, including curvature, connectivity, etc.

as a disordered point set. It contains the point coordinates sampled from the object surface, and sometimes the datasets are with additional features (e.g., normal, color, laser intensity). Compared with other 3D data formats (e.g. non-rational B-spline surface [23] and triangle mesh [4]), the point cloud could be obtained and maintained with fewer efforts because of the simple structure. However, the sparse and disordered nature challenges the geometric feature extraction. Consequently, it is critical and challenging to capture the underlying geometric features, and convert the features into a robust descriptor (i.e., hidden representation).

In general, there are three branches (i.e., voxel-based, multi-view based and point-based branches) to analyze the point cloud. For the convenience of downstream calculation, two of the branches, voxel-based and multi-view based branches, convert the point cloud into a regular data format beforehand. Unfortunately, the simple conversion could cause geometric distortion [39]. Voxel-based approaches [21, 37] leverage 3D grids to represent the point clouds, which results in surface deformation, along with unneces-

sary computing cost [29, 46, 39]. Meanwhile, multi-view based approaches [30, 5] take advantage of the 2D projection images as input, which damages the original 3D geometric structures (e.g., depth, volume) and causes the occlusion issue [39].

Despite its success of the third branch [25, 26, 46], which takes the raw data as input directly. **The long-range manifold features can weakly be characterized still, and the robustness for non-rigid transformations is rarely discussed.** As a critical latent structure in the point cloud, the long-range manifold (e.g., contour) possesses considerable global features (e.g., **curvature, symmetry, similarity**). For example, a ball and a book have contours with different curvatures to be distinguished. **With the global nature, the long-range features are more robust against the large transformations (e.g., rotation, scale, shear). In contrast, the local features may undergo dramatic changes when non-rigid transformations are encountered.** It is significant to capture the global geometric features for non-rigid robustness, since slight non-rigid transformations (e.g., shear) are inevitable in real-scanned point clouds. For instance, handmade clay pots usually have different sizes and distortions. In addition, **the local features could not describe some long-range properties (e.g., symmetry, periodicity of changes in curvature), which are invariant for rigid transformations.** To gather more global features, a few papers [39, 8] have attempted to introduce distant correlations. **Nevertheless, the direct connection among similar remote points can hardly characterize a complete long-range manifold, which limits the model capability to capture the long-range patterns (e.g., length, the regularity of curvature changes).**

To find the meaningful contours, an easy way is to look for the geodesics [3, 11, 7]. However, if we directly search the shortest paths on the point cloud, there will be extremely plenty of geodesics (i.e., combinatorial explosion). **The redundant geodesics could overwhelm the point cloud analysis from the perspectives of both effectiveness and efficiency.** Thus, to solve this critical challenge, we put forward a methodology (i.e., **“Viewpoint Path” mechanism, Fig.4 & 3**) to adaptively construct the contour-based long-range connections with high efficiency.

Based on the viewpoint path mechanism, **we propose the ContourNet, which establishes the contour-based connections to extract the long-range manifold features explicitly.** In Fig.1, compared with the local connection used in the CNN-like models [26, 20], our contour-based connection can provide more global (i.e., long-range) contexts, including curvature, colinearity, connectivity, etc. Specifically, for the neural architecture, we create three variants of ContourNet: ContourNet-V1, ContourNet-V2 and ContourNet-V3, progressively.

In summary, our contributions are as follows:

- We propose a novel neural architecture, ContourNet, which explicitly leverages large-range geometric metrics to analyze the point cloud. The large-range features confer a strong robustness against rigid and non-rigid transformations.
- We introduce **“Viewpoint Path” mechanism** (Fig.3 & 4), which adaptively constructs the contour-based long-range connections on the point cloud. This brings a new path-way towards point cloud analysis in our community.
- Our proposed ContourNet achieves the state-of-the-art performance on the benchmark datasets (i.e., ModelNet40 [37] and ScanObjectNN [32]), which demonstrates the effectiveness of long-range geometric metrics.

2. Related Work

2.1. Voxel-Based & Multi-View Based

Motivated by the success of the traditional convolution in regular data format (e.g., 2D image), some researchers convert the point cloud to other data formats.

Voxel-Based approaches [21, 37] represent the irregular 3D point cloud lossily using 3D grid. Because of the sparsity of the input point cloud, this dense representation brings a lot of extra computational costs. On the one hand, Kd-Net [12] and Oct-Net [28] skip the empty voxels to reduce resource consumption. However, the information loss caused by voxelization is still difficult to avoid [29, 46, 39]. On the other hand, to keep more data details, 3DmFV [43] describes points by their deviation from a Gaussian Mixture Model. But it still retains the large computing cost [45].

Multi-View Based works [30, 5] take the 2D images from multiple perspectives as input. This lower-dimensional representation ignores the intrinsic geometric relationship, and further causes the occlusion problem [46].

Compared with these two branches, our ContourNet takes raw point clouds as input, the contours captured by our model retain the original 3D structures.

2.2. Point-Based

Point-based methods aim to deal directly with the point cloud data. In this branch, various approaches [25, 26, 46, 42, 1, 8, 9, 39, 41, 29, 18, 16, 19, 20, 42, 40] have been proposed and achieved promising results. As a pioneer, PointNet [25] makes use of the symmetric function to aggregate point features, which solves the disordered issue. But it can hardly extract the fine-grained local information, which is vital for the complex scenarios. Inspired by CNN, some models [26, 34] utilize the grouping operation (k-nearest neighbor group or ball region group) to obtain the local features. Nevertheless, the relationships between distant points

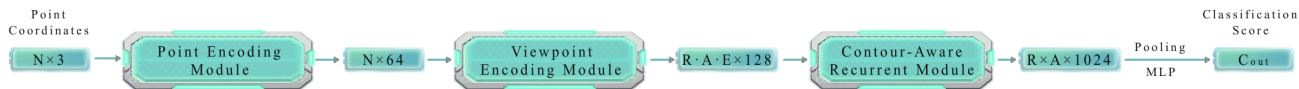


Figure 2. **ContourNet-V1/V2 Architecture for Classification** (Best view in zoom in). “ N ” is the number of points. “ C_{out} ” denotes the number of categories. “ R ”, “ A ”, “ E ” are parameters for viewpoint layout and “ $R \cdot A \cdot E$ ” means the scalar-multiplication result of R , A and E . Given the point coordinates, we first leverage the *Point Encoding Module* (PEM) to enhance the point features. Then, based on the point features, our *Viewpoint Encoding Module* (VEM) is driven to encode the viewpoint features. Next, *Contour-Aware Recurrent Module* (CARM) captures the contour features along the viewpoint paths. Last, the the contour features in the same layer are pooled, and all pooled features of different layers are concatenated to classify.

are neglected. To obtain the non-local information, GS-Net [39] constructs the local connections in Euclidean space and Eigenvalue space respectively. Point2Node [8] introduces the non-local correlation through the attention mechanism. PointASNL [42] adaptively adjusts the sampling, and leverages non-local information to process the point cloud with noise. Besides, there remains a lot of excellent literature, such as [38, 47, 17, 31, 15, 36, 13, 14, 27].

However, most of the previous works focus on the point-based exploration of geometric features. This limits the model capacity to capture the long-range geometric features, which provide a significant global description for the point cloud. Beyond the point-based exploration, RCNet [35] partitions the ambient space into parallel beams, and leverages RNN to process the points within the same beam. Although the beam contains the long-range points, surface connectivity with semantic information is broken. In ContourNet, we can capture a more complete long-range manifold.

3. Methodology

We introduce our neural architecture in Fig.2, Fig.5 and the corresponding subsections. As mentioned before, there are three ContourNet variants: ContourNet-V1, ContourNet-V2 and ContourNet-V3.

3.1. ContourNet-V1

As shown in Fig.2, our ContourNet-V1 contains three core modules, followed by a Multilayer Perceptron (MLP) classifier. The three modules are *Point Encoding Module* (PEM), *Viewpoint Encoding Module* (VEM) and *Contour-Aware Recurrent Module* (CARM). Generally, PEM encodes the features for each point. VEM maps each viewpoint to a local subset of points, and abstracts viewpoint features from the local points. CARM focuses on converting the viewpoint features into contour features.

Point Encoding Module (PEM)

In ContourNet-V1, *Point Encoding Module* (PEM) is a simple Multilayer Perceptron (MLP). Given a point cloud $P = \{p_i \in \mathbb{R}^3, i = 1, 2, \dots, N\}$, where N is the number of points, P indicates the point coordinate set, p_i denotes

the i -th 3D point coordinate. By the point-shared MLP, the point coordinates are mapped into a high-dimensional point feature space.

Viewpoint Encoding Module (VEM)

Point clouds are sparsely sampled from 3D surfaces [39], which can be leveraged to capture the object contours. Thus, we can search the shortest paths (i.e., geodesics) to form contours. However, this method is inefficient for the high-resolution point cloud. Moreover, due to the **different lengths and directions**, the captured contours may have a large variance. The large variance will lead to unsatisfactory performance.

Therefore, in this module, we propose the viewpoint path mechanism, which constructs the long-range estimated contour connections effectively and efficiently. There are three steps in *Viewpoint Encoding Module* (VEM). (1) **Viewpoint Placement** aims to put viewpoints around the point cloud. (2) **Viewpoint Encoding** maps viewpoints to their observation regions (Fig.3), and encodes the viewpoint features from the regions. (3) **Contour-Based Connection** leverages viewpoints in the same path to form the contour.

(1) **Viewpoint Placement.** To build the contour-based connection, we initialize multiple **viewpoint paths**. Each path consists of an array of ordered **viewpoints**, where the “viewpoint” aims to look for a local sub-manifold of the point cloud (Fig.3). We denote this sub-manifold as an “observation region” of the viewpoint, and denote the center of this sub-manifold as an “contour node”. Thus, to form a contour-based long-range manifold (Fig.3), we can sequentially connect the contour nodes, which are observed by those viewpoints in the same path. For better overall coverage, the viewpoint path is designed as a semi-circular arc from top to bottom. The viewpoints on the same path are distributed uniformly, and the viewpoint paths are evenly placed around the z -axis. Besides, to obtain sufficient contour features, we apply the multi-scale technique, which constructs the multi-layer viewpoint paths (Fig.4). In this way, we represent the viewpoint coordinate set as $S = \{s_{r,a,e} \in \mathbb{R}^3, 0 < r \leq R, 0 < a \leq A, 0 < e \leq E\}$, where R stands the number of layers for multiple scales, A presents the number of paths in each layer, E denotes the number of viewpoints for each path, and $s_{r,a,e}$ denotes the

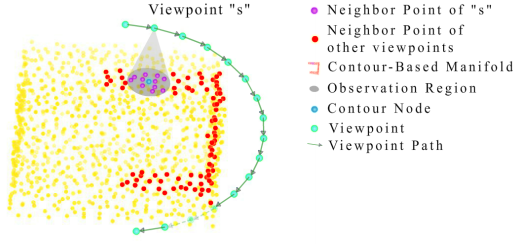


Figure 3. **Viewpoint Path and The Corresponding Contour-Based Manifold** (Best view in color and zoom in). To obtain the estimated contour (Fig.1) and form a long-range manifold (a.k.a. the red points and the purple points), we connect the contour nodes sequentially, which are observed by the viewpoints on the same path.

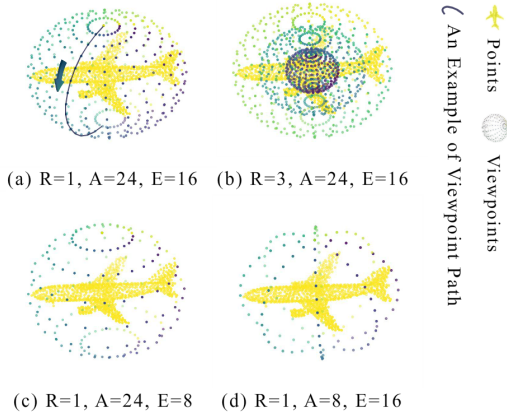


Figure 4. **Initial Viewpoint Path Layouts** (Best view in color and zoom in). R represents the number of layers, A is the number of viewpoint paths in a layer, and E denotes the number of viewpoints on each path.

e -th viewpoint coordinate of the a -th path in the r -th layer. Here, we visualize the effects of some different R, A, E settings in Fig.4.

(2) Viewpoint Encoding. To map each viewpoint to a sub-manifold (i.e., observation region) of the point cloud, we apply the k -nearest neighbor search algorithm, which looks for the k -closest neighbor points $\gamma_{r,a,e} = \{p_j, 0 < j \leq k\}$ for each viewpoint $s_{r,a,e}$, where p_j is the j -th neighbor point, $\gamma_{r,a,e}$ indicates the set of neighbor points for the e -th viewpoint of the a -th path in the r -th layer.

To calculate the local relative features of each neighbor point $p_j, j \in \{1, 2, \dots, k\}$, we take the average neighbor point coordinates as the **contour node** (a.k.a., local center of the observation region) $cen_{r,a,e} \in \mathbb{R}^3$, for the e -th viewpoint of the a -th path in the r -th layer. An example of the contour node can be found in Fig.3.

Formally summarizing, given the neighbor set $\gamma_{r,a,e}$ of

the viewpoint $s_{r,a,e} \in S$. We can obtain the contour node $cen_{r,a,e} \in \mathbb{R}^3$ through Eqs.(1).

$$cen_{r,a,e} = \sum_{j=1}^k \frac{p_j}{k}, p_j \in \gamma_{r,a,e} \quad (1)$$

where $cen_{r,a,e}$ indicates the contour node coordinate observed by the viewpoint $s_{r,a,e}$.

Then, we encode the viewpoint based on the neighbor point features. Given $F_{r,a,e} = \{f_j, 0 < j \leq k\}$, where f_j denotes the high-dimensional point features of j -th neighbor point, and $F_{r,a,e}$ is the neighbor point feature set of viewpoint $s_{r,a,e}$. To obtain the representation $q_{r,a,e}$ of viewpoint $s_{r,a,e}$, we aggregate the point features $F_{r,a,e}$ by Eqs.(2), (3) and (4). Here, $q_{r,a,e} \in Q, Q = \{q_{r,a,e}, 0 < r \leq R, 0 < a \leq A, 0 < e \leq E\}$ indicates the viewpoint feature set.

$$center_feature_{r,a,e} = Mean(F_{r,a,e}) \quad (2)$$

$$edge_j = MLP(\tau(f_j, f_j - center_feature_{r,a,e})) \quad (3)$$

$$q_{r,a,e} = Max(\{edge_j, 0 < j \leq k\}) \quad (4)$$

where $Mean$ and Max indicate the pooling operation over the neighbor points. $edge_j$ means the edge features between the neighbor point p_j and the contour node. τ denotes the concatenation operation along the feature dimension. MLP signifies the Multilayer Perceptron.

To begin with, in Eqs.(2), we average the neighbor features to calculate the center features $center_feature_{r,a,e}$ of the contour node $cen_{r,a,e}$. Next, we abstract the edge features $edge_j$ between the neighbor point $p_j \in \gamma_{r,a,e}$ and the contour node in Eqs.(3). Last, in Eqs.(4), we utilize the point-wise max-pooling, which extracts the viewpoint-specific local geometric features (i.e., observation region features) $q_{r,a,e}$ from the neighbor points of the viewpoint. Notably, this aggregation method is permutation invariant [25] to the neighbor order.

(3) Contour-Based Connection. To draw the contour, we connect the contour nodes of each viewpoint in the same path sequentially. As shown in Fig.1, the contour can describe the general appearance of an object, which is instructive for our model to understand the point cloud at the semantic level.

Contour-Aware Recurrent Module (CARM)

The points are characterized by the spatial coordinates, so the relative position relationships entail the geometric structures. For this reason, we introduce this module to capture the underlying geometric information (e.g. curvature, direction). Formally, we denote the feature set of viewpoint $\{q_{r,a,e}, 0 < e \leq E\}$ in the a -th path of r -th layer as the input sequence $Z_{r,a} = \{z_e = q_{r,a,e}, 0 < e \leq E\}$, where z_e represents the input at sequence step e (a.k.a. the

e -th viewpoint features). We work out the e -th hidden representation h_e , and take the final hidden vector h_E as the contour representation as below.

$$d_e^1 = \begin{cases} 0 & e = 1 \\ z_e - z_{e-1} & e > 1 \end{cases} \quad (5)$$

$$d_e^2 = \begin{cases} 0 & e = 1 \\ d_e^1 - d_{e-1}^1 & e > 1 \end{cases} \quad (6)$$

$$h_e = GRU(\tau(z_e, d_e^1, d_e^2), h_{e-1}) \quad (7)$$

where d_e^1 indicates the gap (i.e., first difference) of features between sequence step e and $e - 1$. If $e = 1$, we set it to 0. d_e^2 represents the difference of the first difference (i.e., second difference). h_e denotes the hidden state in sequence step e . GRU means the cell of GRU, it can be replaced by other RNN cells. τ is the concatenation operation along the feature dimension.

In this way, to obtain the geometric features, *Contour-Aware Recurrent Module (CARM)* can be easily aware of the relative relationship, which is between the adjacent features. By virtue of the gate mechanism [10] of GRU, long-range geometric features can be kept to conclude some sequential geometric properties, such as periodicity of curvature change, colinearity, symmetry. As important intrinsic features, these sequential geometric properties are usually overwhelmed by the CNN-like network, due to the **direct global pooling layer**. On the one hand, the global representation with fuzzy discrimination is worked out, and further **confuses the model to distinguish different samples, which have similar local structures (Fig.6)**. On the other hand, lack of long-range features can also lead to weak robustness for both non-rigid and rigid transformations, since **(1) the non-rigid transformations may cause catastrophic changes to some local structures**, which are regarded as vital clues for the model to classify. **(2) When rigid transformations are encountered, some invariant long-range features (e.g., periodicity of changes in curvature) are difficult to capture.**

3.2. ContourNet-V2

Since some vital contours do not follow the longitude in several specific samples. Given the example in Fig.1, the vital contour nodes emerge in the piano top cover. Inspired by PointNet [25], to capture the crucial contours out of the longitude, we enhance the *PEM* by two fully connected layers with T-Net [25] in ContourNet-V2. The T-Net allows an affine transformation for the input point cloud. Besides, the other modules follow ContourNet-V1.

3.3. ContourNet-V3

In our best developed model (i.e., ContourNet-V3), we design a convolutional kernel to extract the local features

of each point. Since our viewpoints can aggregate point features to enrich the geometric features of contour nodes. Therefore, the method of convolution can be easily utilized by our ContourNet. Furthermore, we employ the attention mechanism to offset the contour nodes observed by viewpoints. Besides, the viewpoint path is allowed to change its shape and position by T-Net [25].

As illustrated in Fig.5, our ContourNet-V3 contains a Master Branch and two complementary sub-branches, which respectively focus on the local and contour-based global features.

Above all, regarding the Head Part of the Master Branch, as shown in Fig.2, we first initialize multiple **viewpoint paths**. Second, we create the Eigen-Graph [39] to obtain the eigenvalue features, and the point coordinates and eigenvalue features are concatenated. Third, to transform the point coordinates and viewpoint path positions, the concatenated features are fed into our *Point-Spatial Transform Module (PSTM)*. Last, the adjusted coordinates are put into the following two sub-branches (i.e., Local Representation Branch & Global Representation Branch).

Subsequently, the Local Representation Branch stacks two our *Space Convolution* layers to hierarchically extract the local features within both Euclidean and Eigenvalue space [39]. The output of each convolution layer is concatenated and fed to a fully connected layer, in which the parameters are shared by points. To aggregate the local information from all the points, the point-wise max-pooling and mean-pooling are applied to the output of the fully connected layer.

Next, the Global Representation Branch focuses on the contour feature extraction. This branch has a similar architecture to ContourNet-V2. It is worth noting that we modify from two different points. (1) We propose the *Viewpoint Locating Module (VLM)* to offset the contour node of each viewpoint. Specifically, according to the distance from the viewpoint to the neighbor, we assign weight to each neighbor. Thus, we can obtain the offset center features. (2) The viewpoint feature extraction is based on the high-dimensional point features and the output of the last convolution layer. By this design, it confers a capability to extract additional contextual information (e.g. local structural symmetry) on our *Contour-Aware Recurrent Module (CARM)*.

Eventually, regarding the Tail Part of the Master Branch, we concatenate the outputs of the Local and Global Representation Branches as the final representation. Then, the final representation is leveraged to classify by a Multi-Layer Perceptron, or regarded as an additional global context for other tasks (e.g., segmentation).

For more details about ContourNet-V3, please refer to the supplementary materials, in which we provide a module table and a more specific description.

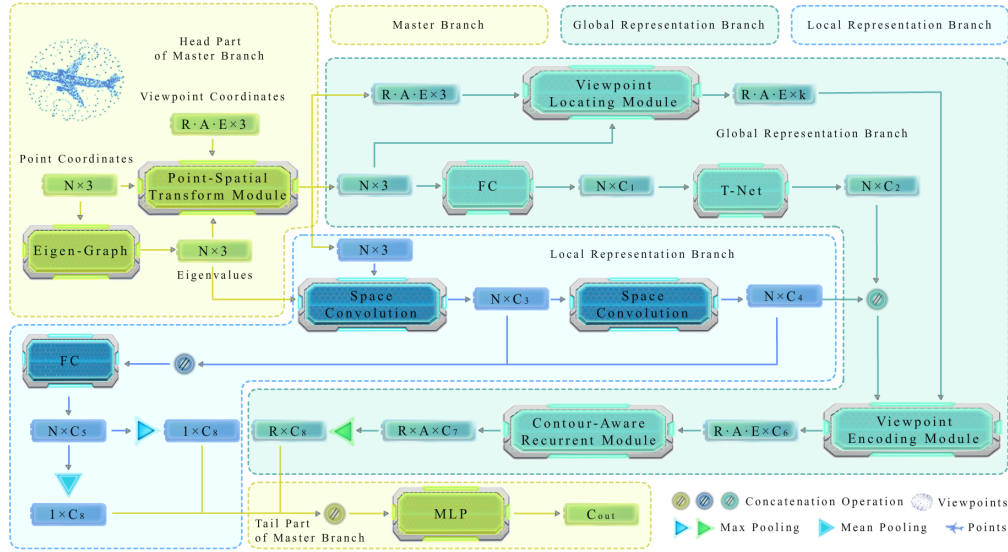


Figure 5. **ContourNet-V3 Architecture for Classification** (Best view in color and zoom in). “ $C_1, C_2, \dots, C_8$ ” denote the feature channels. “ C_{out} ” is the category number. The cyan part represents the Global Representation Branch, and the blue part denotes the Local Representation Branch. They are sub-branches of the Master Branch (i.e., the yellow-green part). First, by the input point coordinates and the eigenvalues extracted by Eigen-Graph, we employ our *Point-Spatial Transform Module (PSTM)* to transform the viewpoint position and the point coordinates. Second, the Local Representation Branch encodes the local geometric features, and integrates them to generate the summative local representation. In the Global Representation Branch, to obtain the viewpoint features in *Viewpoint Encoding Module (VEM)*, the local information and the high-dimensional point features are leveraged. Then, the *Contour-Aware Recurrent Module (CARM)* abstracts the global contour features. Finally, the concatenation of the local and global representations goes through a Multi-Layer Perceptron (MLP) to classify.

4. Experiments

4.1. Point Cloud Classification

4.1.1 ModelNet40

ModelNet40 [37] is a point cloud classification benchmark. This dataset consists of 9,843 training samples and 2,468 test samples marked for 40 categories. In the training phase, we take 1,024 points as input, which are uniformly sampled from each CAD model [37]. Besides, we report the result with voting trick to average the predictions, which follows previous works [20, 26].

Tab.1 shows the quantitative comparisons of the performance between our ContourNet and the recent advances. In the case of the same input (i.e., 1,024 points), our ContourNet achieves the state-of-the-art performance. Moreover, our model exceeds the methods with extra input. This result clarifies the effectiveness of ContourNet for point cloud classification.

4.1.2 ScanObjectNN

ScanObjectNN [32] is a more challenging classification dataset, which focuses on real-world objects. This bench-

mark includes the point cloud data of 15,000 objects classified into 15 categories, and we use its two representative variants (i.e., OBJECT_ONLY and PB_T50_RS) to further evaluate our ContourNet. The PB_T50_RS subset is more difficult due to the additional perturbations.

Tab.2 reports the results of ContourNet on ScanObjectNN [32], measured by overall accuracy. Similar to ScanObjectNN [32], we do not apply the transformations other than jitter and rotation to augment the training data. Our ContourNet achieves the best results on both two variants of the dataset. This proves that even if the dataset contains more hard cases, our model can still analyze the point cloud well. The point cloud data from the real world further demonstrates the industrial value of ContourNet.

4.2. Part Segmentation

Given that our main contribution is to extract global geometric features, semantic segmentation is too fine-grained to verify the effectiveness of the large-range manifold features directly. For this purpose, the classification is more reasonable. However, it is easy to leverage our contour features for other tasks. Specifically, inspired by PointNet [25], we concatenate the output of the Global Representation Branch to each point representation, which extracted

Method	#points	Acc.(%)
The below baselines take 3D spatial coordinates as input.		
PointNet[25]	1k	89.2
PointNet++[26]	1k	90.7
PointCNN[18]	1k	92.2
DGCNN[34]	1k	92.2
DGCNN*[34]	1k	92.9
Point2Sequence[19]	1k	92.6
RCNet-E[35]	1k	92.3
RS-CNN[20]	1k	93.6
DensePoint[20]	1k	93.2
Grid-GCN[40]	1k	93.1
Point2Node[8]	1k	93.0
GS-Net[39]	1k	92.9
PointASNL[42]	1k	93.2
GLBR w/ PN++ (Large)[27]	1k	93.0
ContourNet-V1 (ours)	1k	92.3
ContourNet-V2 (ours)	1k	93.2
ContourNet-V3 (ours)	1k	94.0
The below baselines take 3D spatial coordinates and normal vectors as input.		
PointNet++[26]	5k	91.9
SpiderCNN[41]	1k	92.4
SO-Net[16]	5k	93.4
PointWeb[46]	1k	92.3
PointConv[36]	1k	92.5
Geo-CNN[14]	1k	93.4

Table 1. **Shape Classification Results on ModelNet40.** “#points” column shows the input point number. “Acc.” denotes the overall accuracy. “DGCNN*” represents the new version of DGCNN published in TOG [34].

Method	OBJ_ONLY	PB_T50_RS
PointNet[25]	79.2	68.2
PointNet++[26]	84.3	77.9
3DmFV[43]	73.8	63.0
SpiderCNN[41]	79.5	73.7
PointCNN[18]	85.5	78.5
DGCNN*[34]	86.2	78.1
GS-Net[39]	84.7	78.1
ContourNet-V3 (ours)	86.6	80.8

Table 2. **Overall Accuracy(%) on ScanObjectNN.** Notably, this comparison only includes the supervised models, which take the ScanObjectNN as the unique training dataset.

by other SOTA baselines. The results on ShapeNet Part [2] are as shown in Tab.5. To fairly evaluate the validity of contour features, we retrained the SOTA baselines to ensure that the models with/without the contour features have the

Method	z/z	0/z	s/s	0/s	e/e
PointNet[25]	81.6	55.2	66.3	11.7	88.6
DGCNN[34]	90.4	65.7	82.6	19.6	91.3
GS-Net[39]	89.8	69.8	87.9	19.4	90.9
GS-Net($\lambda_j - \lambda_i, \lambda_j$)[39]	85.0	-	82.8	50.7	-
ContourNet-V3 (ours)	91.9	76.8	88.3	16.3	92.4

Table 3. **Robustness Analysis for ContourNet-V3.** The numbers in the table represent the overall accuracy on ModelNet40.

Method	Forward Time(ms)	Model Size(M)	Accuracy (%)
PointNet[25]	30	13.4	89.2
PointNet++[26]	603	7.0	91.9
DGCNN[34]	73	7.2	92.2
GS-Net[39]	126	6.0	92.9
ContourNet-V1 (ours)	40	8.0	92.3
ContourNet-V2 (ours)	45	18.2	93.2
ContourNet-V3 (ours)	68	34.6	94.0

Table 4. **Efficiency Analysis of Shape Classification on ModelNet40.**

Method	/wo Contour Features	/w Contour Features	Gain
PointNet[25]	82.37	82.70	+0.33
DGCNN[34]	84.00	84.16	+0.16
GS-Net[39]	82.83	83.80	+0.97

Table 5. **Instance mIoU on ShapeNet Part.** Here we assume that the class of each sample is unknown and sample 1,024 points for each point cloud. “/wo Contour Features” denotes our contour features are not concatenated, while “/w Contour Features” means the contour features are leveraged.

R	A	E	1st diff	2nd diff	Voting	OA(%)
3	32	16				92.3
3	32	16	✓			92.7
3	32	16	✓	✓		92.8
2	32	16	✓	✓		92.5
4	32	16	✓	✓		93.0
3	30	16	✓	✓		92.4
3	34	16	✓	✓		92.4
3	32	14	✓	✓		92.7
3	32	18	✓	✓		92.7
4	32	16	✓	✓	✓	93.2

Table 6. **“R, A, E” Hyper-Parameter Sensitivity Experiment and Ablation Study for CARM in ContourNet-V2.**

same settings. Notably, the Global Representation Branch is not fine-tuned.

Experimental results (Tab.5) demonstrate that our contour features bring significant improvements for the segmentation task. Notably, without the class scores, there are no direct global features for GS-Net. Thus, our contour features perform best (+0.97) for GS-Net.

4.3. Robustness Analysis

We conduct robustness tests on ModelNet40 [37]. The results are reported in Tab.3 with five comparisons. (1) Both training set and test set are augmented by random angle rotation for z -axis (z/z); (2) Only test set with rotation for z -axis ($0/z$); (3) Both the training set and test set are augmented by rotation for all axes (s/s); (4) Only test set with rotation for all axes ($0/s$). (5) Both training set and test set are augmented by random shear (e/e). Notably, we leverage the shear augmentation code of PyTorch Geometric [6] with maximal factor 0.5. Meanwhile, in this experiment, we do not conduct the voting technique.

Tab.3 reveals the strong robustness of our ContourNet. Our model achieves the best performance in tests (z/z), ($0/z$), (s/s) and (e/e). In addition, we observe that ContourNet does not perform well in test ($0/s$). It is caused by the relatively fixed direction (i.e., from top to bottom) of viewpoint paths. In test phase, due to rotations around the X -axis or Y -axis, the captured path set may be different. Nevertheless, we can easily solve this issue by rotation augmentation, which leads the model to learn multiple possible path sets. With simple rotation augmentation in the training phase, our model demonstrates excellent effectiveness in test (s/s). Moreover, in the real world, the rotation around z -axis is much more common, while an upside-down object is rare. As our ContourNet achieves outstanding performance in test ($0/z$), it can offer great practicability.

4.4. Efficiency Analysis

We assess the time efficiency of our ContourNet on the ModelNet40 benchmark [37]. All the reported models in Tab.4 are implemented by PyTorch [22] and this experiment is conducted on a single GTX 1080 GPU. The forward time is recorded with a batch size of 8, while we have considered the k -NN process in this testing. The codes for our k -NN process are copied from GS-Net [39]. In short, our ContourNet needs an acceptable forward time while achieving the best accuracy in these models. Notably, most of our model parts can be executed parallel, thus with a relatively large parameter scale, our model can still run fast.

4.5. Ablation Study and Hyper-Parameter Sensitivity Experiment

We conduct the our analyses on the ModelNet40 benchmark [37]. In Tab.7, regarding our ContourNet-V3, our global contour features bring 0.8% overall accuracy improvement, and the adaptive modules can enhance the capability of capturing essential contours (0.3%).

We also employ some specific tests for our *Contour-Aware Recurrent Module (CARM)*, the tests are about ContourNet-V2. In Tab.6, the “1st diff” signifies whether the first difference are concatenated in Eqs.7. As core de-

Local	Global	Adaptive Center	RNN Cell	Voting	Accuracy (%)
✓					92.6
	✓		GRU		93.1
✓	✓		GRU		93.4
✓	✓	✓	GRU		93.7
✓	✓	✓	LSTM		93.0
✓	✓	✓	GRU	✓	94.0

Table 7. **Ablation Study for ContourNet-V3.** “Local” represents the pooling results (i.e., two $1 \times C_8$ tensors) of the Local Representation Branch, while “Global” indicates the output (i.e., $R \times C_8$ tensor) of our Global Representation Branch.

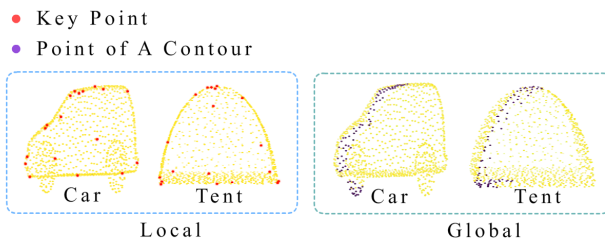


Figure 6. **Visualization of key points and contours** (Best view in color and zoom in).

sign of our *CARM*, the first and second differences offer a considerable improvement.

Besides, to test the sensitivity of our contour extraction modules, we adjust the hyper-parameters R , A and E of ContourNet-V2. As shown in Tab.6, our model are insensitive to the hyper-parameters.

4.6. Visualization

In Fig.6, we visualize a failure case of our baseline with only Local Representation Branch, where the model incorrectly predicts the car as a tent. The car and tent have similar local structures at many key points, which confuse the model to distinguish the car and tent. On the contrary, our contour can provide a large-range pattern with sequential curvature changes, which helps the ContourNet successfully identify the car.

5. Conclusion

We propose a novel neural architecture called ContourNet, which establishes the contour-based connections to analyze the point cloud. The contour features provide a more detailed global description, while they are more robust against the large geometric transformations. In our proposed network architecture, we design three variants of ContourNet with different capabilities and efficiencies, and our model ContourNet-V3 achieves the state-of-the-art performance on the benchmark datasets.

References

- [1] Matan Atzmon, Haggai Maron, and Yaron Lipman. Point Convolutional Neural Networks by Extension Operators. *arXiv preprint arXiv:1803.10091*, 2018. 2
- [2] Angel X Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, et al. ShapeNet: An Information-Rich 3D Model Repository. *arXiv preprint arXiv:1512.03012*, 2015. 7
- [3] Edsger Wybe Dijkstra. A Note on Two Problems in Connexion with Graphs. In *Numerische Mathematik*, pages 269–271, 1959. 2
- [4] Yutong Feng, Yifan Feng, Haoxuan You, Xibin Zhao, and Yue Gao. MeshNet: Mesh Neural Network for 3D Shape Representation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 8279–8286, 2019. 1
- [5] Yifan Feng, Zizhao Zhang, Xibin Zhao, Rongrong Ji, and Yue Gao. GVCNN: Group-View Convolutional Neural Networks for 3d Shape Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 264–272, 2018. 2
- [6] Matthias Fey and Jan Eric Lenssen. Fast Graph Representation Learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428*, 2019. 8
- [7] Robert W Floyd. Algorithm 97: Shortest Path. In *Communications of the ACM*, pages 345–345, 1962. 2
- [8] Wenkai Han, Chenglu Wen, Cheng Wang, Xin Li, and Qing Li. Point2Node: Correlation Learning of Dynamic-Node for Point Cloud Feature Modeling. *arXiv preprint arXiv:1912.10775*, 2019. 2, 3, 7
- [9] Tong He, Haibin Huang, Li Yi, Yuqian Zhou, Chihao Wu, Jue Wang, and Stefano Soatto. GeoNet: Deep Geodesic Networks for Point Cloud Analysis. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6888–6897, 2019. 2
- [10] S. Hochreiter and J Schmidhuber. Long Short-Term Memory. In *Neural Computation*, pages 1735–1780, 1997. 5
- [11] Donald Bruce Johnson. Efficient Algorithms for Shortest Paths in Sparse Networks. In *Journal of the ACM (JACM)*, pages 1–13, 1997. 2
- [12] Roman Klokov and Victor Lempitsky. Escape from Cells: Deep Kd-Networks for the Recognition of 3D Point Cloud Models. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 863–872, 2017. 2
- [13] Artem Komarichev, Zichun Zhong, and Jing Hua. A-CNN: Annularly Convolutional Neural Networks on Point Clouds. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7421–7430, 2019. 3
- [14] Shiyi Lan, Ruichi Yu, Gang Yu, and Larry S Davis. Modeling Local Geometric Structure of 3D Point Clouds using Geo-CNN. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 998–1008, 2019. 3, 7
- [15] Itai Lang, Asaf Manor, and Shai Avidan. SampleNet: Differentiable Point Cloud Sampling. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. 3
- [16] Jiaxin Li, Ben M Chen, and Gim Hee Lee. So-Net: Self-Organizing Network for Point Cloud analysis. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, pages 9397–9406, 2018. 2, 7
- [17] Ruihui Li, Xianzhi Li, Pheng-Ann Heng, and Chi-Wing Fu. PointAugment: an Auto-Augmentation Framework for Point Cloud Classification. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. 3
- [18] Yangyan Li, Rui Bu, Mingchao Sun, Wei Wu, Xinhan Di, and Baoquan Chen. PointCNN: Convolution On X-transformed Points. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 820–830, 2018. 2, 7
- [19] Xinhai Liu, Zhizhong Han, Yu-Shen Liu, and Matthias Zwicker. Point2Sequence: Learning the Shape Representation of 3d Point Clouds with an Attention-based Sequence to Sequence Network. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 8778–8785, 2019. 2, 7
- [20] Yongcheng Liu, Bin Fan, Shiming Xiang, and Chunhong Pan. Relation-Shape Convolutional Neural Network for Point Cloud Analysis. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8895–8904, 2019. 2, 6, 7
- [21] Daniel Maturana and Sebastian Scherer. VoxNet: A 3d Convolutional Neural Network for Real-Time Object Recognition. In *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 922–928, 2015. 1, 2
- [22] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in PyTorch. In *Advances in Neural Information Processing Systems (NIPS)*, pages 8024–8035, 2017. 8
- [23] Les Piegl and Wayne Tiller. B-spline Curves and Surfaces. In *The NURBS Book*, pages 81–116, 1997. 1
- [24] Charles R Qi, Wei Liu, Chenxia Wu, Hao Su, and Leonidas J Guibas. Frustum Pointnets for 3D Object Detection from RGB-D Data. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, pages 918–927, 2018. 1
- [25] Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, pages 652–660, 2017. 2, 4, 5, 6, 7
- [26] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J Guibas. PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space. In *Advances in neural information processing systems (NeurIPS)*, pages 5099–5108, 2017. 2, 6, 7
- [27] Yongming Rao, Jiwen Lu, and Jie Zhou. Global-Local Bidirectional Reasoning for Unsupervised Representation Learning of 3D Point Clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5376–5385, 2020. 3, 7
- [28] Gernot Riegler, Ali Osman Ulusoy, and Andreas Geiger. OctNet: Learning Deep 3D Representations at High Resolutions. In *2017 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. 2

- [29] Yiru Shen, Chen Feng, Yaoqing Yang, and Dong Tian. Mining Point Cloud Local Structures by Kernel Correlation and Graph Pooling. In *Proceedings of the IEEE conference on computer vision and pattern recognition(CVPR)*, pages 4548–4557, 2018. 2
- [30] Hang Su, Subhansu Maji, Evangelos Kalogerakis, and Erik Learned-Miller. Multi-view Convolutional Neural Networks for 3D Shape Recognition. In *Proceedings of the IEEE international conference on computer vision(CVPR)*, pages 945–953, 2015. 2
- [31] Weiwei Sun, Wei Jiang, Eduard Trulls, Andrea Tagliasacchi, and Kwang Moo Yi. Attentive Context Normalization for Robust Permutation-Equivariant Learning. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. 3
- [32] Mikaela Angelina Uy, Quang-Hieu Pham, Binh-Son Hua, Thanh Nguyen, and Sai-Kit Yeung. Revisiting Point Cloud Classification: A New Benchmark Dataset and Classification Model on Real-World Data. In *Proceedings of the IEEE International Conference on Computer Vision(CVPR)*, pages 1588–1597, 2019. 2, 6
- [33] Dominic Zeng Wang and Ingmar Posner. Voting for Voting in Online Point Cloud Object Detection. In *Robotics: Science and Systems*, pages 10–15607, 2015. 1
- [34] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E Sarma, Michael M Bronstein, and Justin M Solomon. Dynamic Graph CNN for Learning on Point Clouds. In *Acm Transactions On Graphics (tog)*, pages 1–12, 2019. 2, 7
- [35] Pengxiang Wu, Chao Chen, Jingru Yi, and Dimitris Metaxas. Point Cloud Processing via Recurrent Set Encoding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 5441–5449, 2019. 3, 7
- [36] Wenxuan Wu, Zhongang Qi, and Li Fuxin. PointConv: Deep Convolutional Networks on 3D Point Clouds. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition(CVPR)*, pages 9621–9630, 2019. 3, 7
- [37] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 3D ShapeNets: A Deep Representation for Volumetric Shapes. In *Proceedings of the IEEE conference on computer vision and pattern recognition(CVPR)*, pages 1912–1920, 2015. 1, 2, 6, 8
- [38] Saining Xie, Jiatao Gu, Demi Guo, Charles R. Qi, Leonidas Guibas, and Or Litany. PointContrast: Unsupervised Pre-training for 3D Point Cloud Understanding. In *European Conference on Computer Vision (ECCV)*, 2020. 3
- [39] Mingye Xu, Zhipeng Zhou, and Yu Qiao. Geometry Sharing Network for 3D Point Cloud Classification and Segmentation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 12500–12507, 2020. 1, 2, 3, 5, 7, 8
- [40] Qiangeng Xu, Xudong Sun, Cho-Ying Wu, Panqu Wang, and Ulrich Neumann. Grid-GCN for Fast and Scalable Point Cloud Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition(CVPR)*, pages 5661–5670, 2019. 2, 7
- [41] Yifan Xu, Tianqi Fan, Mingye Xu, Long Zeng, and Yu Qiao. SpiderCNN: Deep Learning on Point Sets with Parameterized Convolutional Filters. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 87–102, 2018. 2, 7
- [42] Xu Yan, Chaoda Zheng, Zhen Li, Sheng Wang, and Shuguang Cui. PointASNL: Robust Point Clouds Processing using Nonlocal Neural Networks with Adaptive Sampling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition(CVPR)*, pages 5589–5598, 2020. 2, 3, 7
- [43] Ben Shabat Yizhak, Lindenbaum Michael, Fischer Anath, et al. 3DmFV: 3D Point Cloud Classification in Real-Time using Convolutional Neural Network. *IEEE Robotics & Automation Letters*, pages 1–1, 2018. 2, 7
- [44] Kun Zhang, Shiquan Qiao, and Kai Gao. A New Point Cloud Reconstruction Algorithm Based-on Geometrical Features. In *2015 7th International Conference on Modelling, Identification and Control (ICMIC)*, pages 1–6, 2015. 1
- [45] Zhiyuan Zhang, Binh-Son Hua, and Sai-Kit Yeung. ShellNet: Efficient Point Cloud Convolutional Neural Networks using Concentric Shells Statistics. In *2019 IEEE International Conference on Computer Vision (ICCV)*, 2019. 2
- [46] Hengshuang Zhao, Li Jiang, Chi Wing Fu, and Jiaya Jia. PointWeb: Enhancing Local Neighborhood Features for Point Cloud Processing. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5565–5573, 2019. 2, 7
- [47] Yongheng Zhao, Tolga Birdal, Jan Eric Lenssen, Emanuele Menegatti, Leonidas Guibas, and Federico Tombari. Quaternion Equivariant Capsule Networks for 3D Point Clouds. In *European Conference on Computer Vision (ECCV)*, 2020. 3